Feasibility demo: morpho-phonology in a speech model's word space, and a past-tense pilot

CAS 784, Group 3 - Morph-2-Vec (Bart, Tanya, Nigel), Milestone 3.

We reproduce the main analogy results of Gauthier, Breiss, Leonard & Chang (2025), *Emergent morpho-phonological representations in self-supervised speech models* (EMNLP 2025). Then we pilot our own question on the same data: **does the model represent the regular English past tense with one shared direction across [t] / [d] / [ɪd]?**

Every number is the rank of the correct word among 334,008 spoken word tokens (0 = exactly right; lower is better). We report the mean rank (the paper's metric) and top-1 accuracy (share of trials with rank 0), with 95% confidence intervals from resampling target words.

In [1]:
```
import sys, json
from pathlib import Path

# find the repo (or share-bundle) root from wherever the notebook is run
here = Path.cwd().resolve()
REPO = next(p for p in [here, *here.parents] if (p /
"scripts/feasibility/common.py").exists())
sys.path[:0] = [str(REPO / "scripts/feasibility"), str(REPO / "scripts")]

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm
from IPython.display import display, Markdown

import common as C
from summarize_analogy_results import unambiguous_results

C.set_style()
pd.set_option("display.precision", 3, "display.width", 160)

results = pd.concat([pd.read_parquet(C.DATA / f"results_{m.lower()}.parquet") for m
in C.MODELS], ignore_index=True)
vbd = pd.read_parquet(C.DATA / "vbd_trials.parquet")
trainer_log = pd.read_csv(C.DATA / "trainer_log.csv")
spec_summary = json.loads((C.DATA / "spec_summary.json").read_text())
```

1. Setup and data

- **Model.** We use the raw hidden states of layer 8 of `facebook/wav2vec2-base`. The paper calls this model **Wav2Vec**.

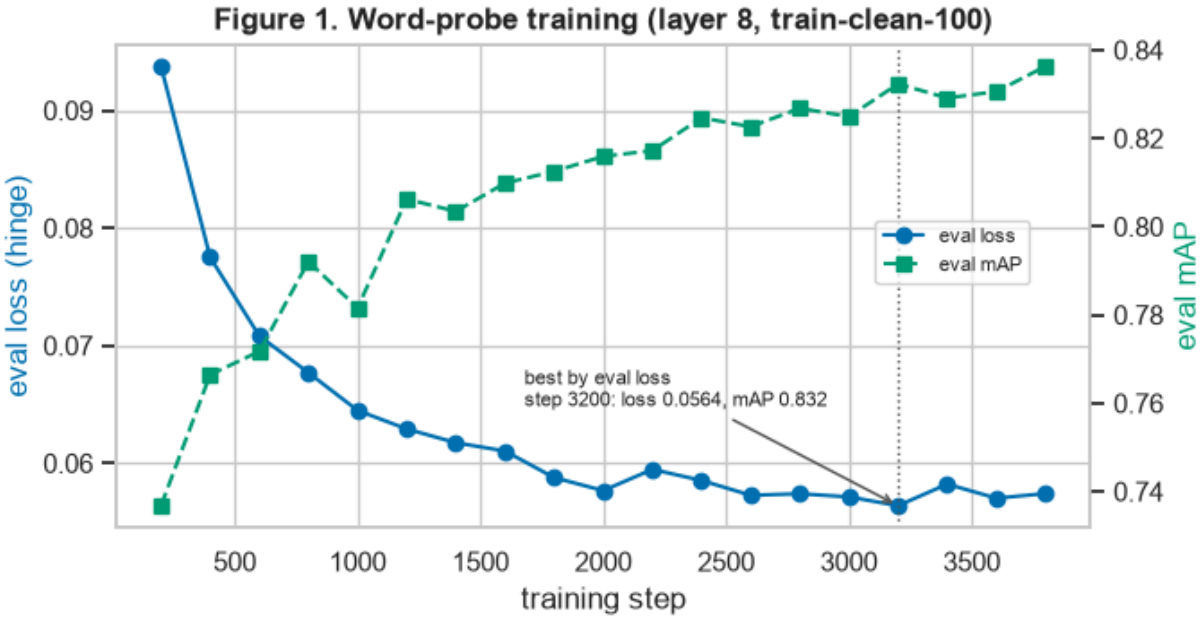
- **Probe.** We also use a word probe: a linear map from the 768-d states down to 32 dimensions. It is trained only to tell spoken words apart (contrastive hinge loss, the paper's Table 9 hyperparameters). The paper calls it **Word**.
- **Data.** LibriSpeech `train-clean-100`: 100.5 h of read English, 28,538 utterances, 18.1M frames of 20 ms. Each spoken word token is represented by the mean of its frames.
- **Analogies.** A trial predicts *d* from *a : b :: c : ?* as b − a + c. We then ask where the true *d* lands among all 334,008 tokens.
- **What to look at in the table:** our analogy universe has exactly the paper's number of tokens.

In [2]:
```
n_nns, n_vbz = spec_summary["unambiguous_pairs"]["NNS"],
spec_summary["unambiguous_pairs"]["VBZ"]
evals = trainer_log.dropna(subset=["eval_loss"])
best = evals.loc[evals.eval_loss.idxmin()]
sanity = pd.DataFrame([
    ("Audio (train-clean-100)", f"{C.AUDIO_HOURS} h, {C.N_UTTERANCES:,}
utterances", "100 h"),
    ("Frames (20 ms)", f"{spec_summary['total_num_frames']:,}", "—"),
    ("Word tokens in the analogy universe (≤ 100 per type)", f"
{spec_summary['n_tokens']:,} ({spec_summary['n_labels']:,} types)", f"
{C.PAPER_N_TOKENS:,}"),
    ("Unambiguous nouns / verbs (regular inflection)", f"{n_nns} / {n_vbz}
({spec_summary['unambiguous_VBZ_after_exclusions']} verbs after the authors'
exclusions)",
     f"{C.PAPER_N_UNAMBIGUOUS[0]} / {C.PAPER_N_UNAMBIGUOUS[1]} (after manual
exclusion)"),
    ("Probe", "linear 768 → 32, 24,576 weights + 32 biases", "same"),
    ("Probe hyperparameters", "margin 0.3759, lr 1.08e-3, weight decay 6.07e-3",
"Table 9: same"),
    ("Probe eval mAP at the best checkpoint", f"{best.eval_mAP:.3f} (step
{int(best.step)})", "not reported"),
], columns=["quantity", "ours", "paper"])
C.save_table(sanity, "table1_sanity")
sanity.style.hide(axis="index")
```

Out[2]:

quantity	ours	paper
Audio (train-clean-100)	100.49 h, 28,538 utterances	100 h
Frames (20 ms)	18,087,600	—
Word tokens in the analogy universe (≤ 100 per type)	334,008 (32,046 types)	334,008
Unambiguous nouns / verbs (regular inflection)	776 / 59 (53 verbs after the authors' exclusions)	753 / 52 (after manual exclusion)
Probe	linear 768 → 32, 24,576 weights + 32 biases	same
Probe hyperparameters	margin 0.3759, lr 1.08e-3, weight decay 6.07e-3	Table 9: same
Probe eval mAP at the best checkpoint	0.832 (step 3200)	not reported

In [3]:
```
fig, ax1 = plt.subplots(figsize=(7, 3.5))
ax2 = ax1.twinx()
ax1.plot(evals.step, evals.eval_loss, "o-", color=C.MODEL_COLORS["Word"],
label="eval loss")
ax2.plot(evals.step, evals.eval_mAP, "s--", color="#009E73", label="eval mAP")
ax1.axvline(best.step, color="#555555", lw=1, ls=":")
ax1.annotate(f"best by eval loss\nstep {int(best.step)}: loss {best.eval_loss:.4f},
mAP {best.eval_mAP:.3f}",
             (best.step, best.eval_loss), xytext=(-150, 40), textcoords="offset
points", fontsize=8,
             arrowprops={"arrowstyle": "->", "color": "#555555"})
ax1.set_xlabel("training step")
ax1.set_ylabel("eval loss (hinge)", color=C.MODEL_COLORS["Word"])
ax2.set_ylabel("eval mAP", color="#009E73")
ax2.grid(False)
ax1.set_title("Figure 1. Word-probe training (layer 8, train-clean-100)")
fig.legend(loc="center right", bbox_to_anchor=(0.86, 0.55), fontsize=8)
C.save_fig(fig, "fig01_probe_training")
plt.show()
```



Training stopped early after three evaluations without an improvement in eval loss, and the best checkpoint (step 3,200) was reloaded. It took about 25 minutes on an RTX 3080 Ti.

- Blue (left axis): eval loss, the penalty on frames the probe never trained on. It falls quickly from 0.094 and levels off around 0.057. Lower is better.
- Green (right axis): eval mAP, how well words can be told apart from the probe's output. It rises from 0.74 to 0.83. Higher is better.
- The dotted line marks the best checkpoint, step 3,200 (lowest eval loss 0.0564, mAP 0.832).

2. Reproduction of the paper's Figs. 3 and 4

The paper's analogies use 753 nouns and 52 verbs whose inflected forms are unambiguously a plural noun (NNS) or a 3rd-person verb (VBZ).

- **Fig. 3** asks whether "add the inflection" transfers across morphological categories (noun → verb).
- **Fig. 4** asks whether it transfers across the pronunciations [s] / [z] / [ɪz].

As in the paper, we drop 6 verbs whose forms sound like another word (*knows* ~ *nose*, *meets* ~ *meats*, …).

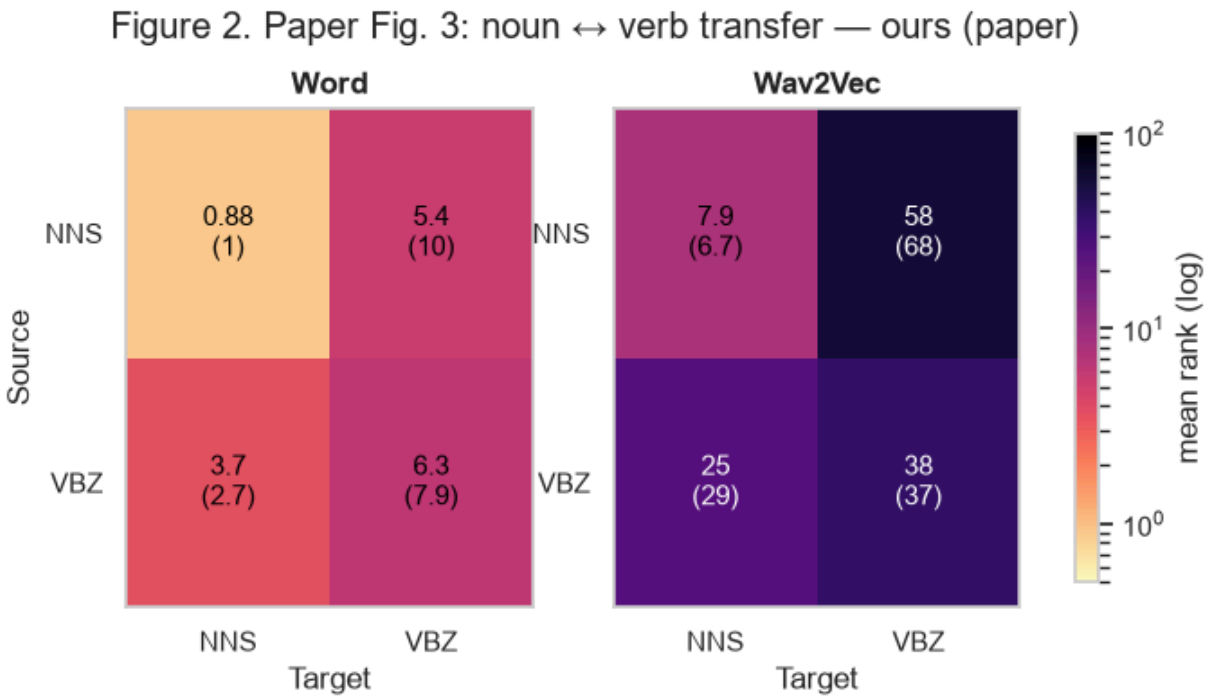
In [4]:
```
u = unambiguous_results(results)
u_excl = u[~u.excluded]
s3 = C.rank_summary(u_excl, ["model", "src_inflection", "tgt_inflection"])
unexcl = u.groupby(["model", "src_inflection",
"tgt_inflection"]).gt_label_rank.mean().rename("mean_rank_no_exclusions")
s3 = s3.merge(unexcl.reset_index(), on=["model", "src_inflection",
"tgt_inflection"])
s3["paper_mean_rank"] = [C.PAPER_FIG3[(m, a, b)] for m, a, b in zip(s3.model,
s3.src_inflection, s3.tgt_inflection)]
C.save_table(s3, "table2_fig3_reproduction")
cols = ["model", "src_inflection", "tgt_inflection", "paper_mean_rank",
"mean_rank_no_exclusions", "mean_rank", "mean_lo", "mean_hi",
        "median_rank", "top1", "n_trials", "n_items"]
s3[cols].style.hide(axis="index").format(precision=2)
```

Out[4]:

model	src_inflection	tgt_inflection	paper_mean_rank	mean_rank_no_exclusions	mean_rar
Wav2Vec	NNS	NNS	6.70	7.87	7.8
Wav2Vec	NNS	VBZ	68.00	51.24	58.(
Wav2Vec	VBZ	NNS	29.00	21.72	24.9
Wav2Vec	VBZ	VBZ	37.00	36.29	38.(
Word	NNS	NNS	1.00	0.88	0.8
Word	NNS	VBZ	10.00	5.35	5.4
Word	VBZ	NNS	2.70	3.24	3.6
Word	VBZ	VBZ	7.90	5.87	6.2

In [5]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 3.6))
norm = LogNorm(vmin=0.5, vmax=100)
for ax, model in zip(axs, C.MODELS):
    s = s3[s3.model == model].pivot(index="src_inflection",
columns="tgt_inflection", values="mean_rank")
    p = s3[s3.model == model].pivot(index="src_inflection",
columns="tgt_inflection", values="paper_mean_rank")
    annot = s.map(C.fmt_rank) + "\n(" + p.map(lambda v: f"{v:g}") + ")"
    im = C.heatmap(ax, s, annot, norm, title=model, fontsize=11, ylabel="Source" if
model == C.MODELS[0] else "")
fig.colorbar(im, ax=axs, label="mean rank (log)", shrink=0.9)
```

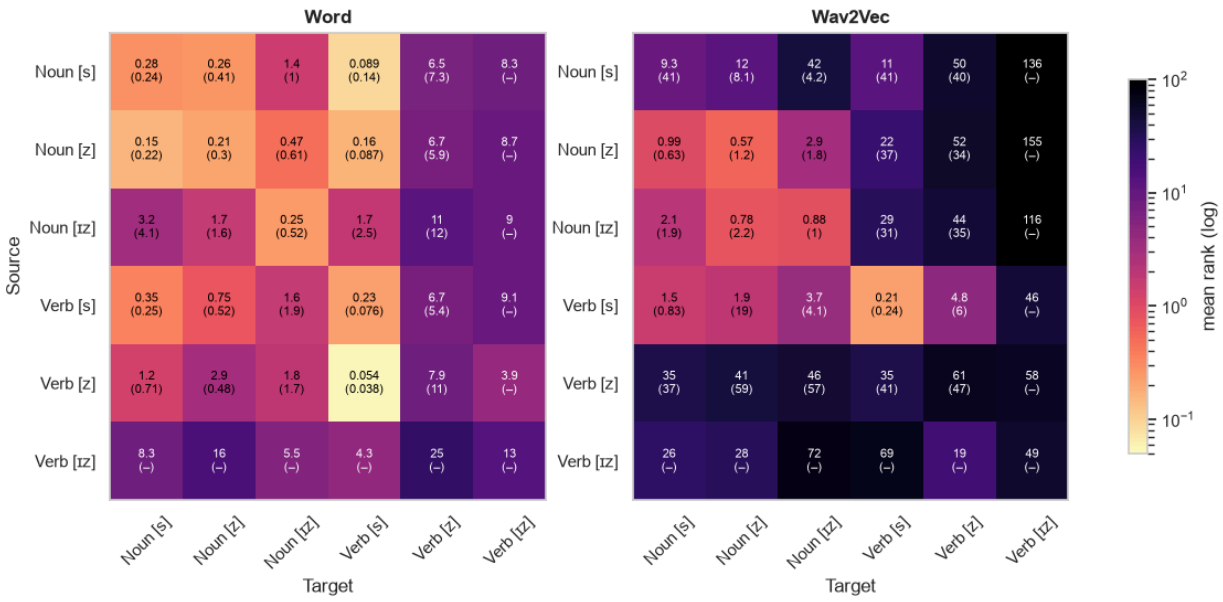
```
fig.suptitle("Figure 2. Paper Fig. 3: noun ⇔ verb transfer — ours (paper)", x=0.45,
y=1.03)
C.save_fig(fig, "fig02_fig3_morphology")
plt.show()
```



In [6]:

```
grid = u_excl.assign(src=lambda d: d.src_inflection.map({"NNS": "Noun", "VBZ":
"Verb"}) + " [" + d.src_allomorph.map(C.ALLOMORPH_IPA) + "]",
                     tgt=lambda d: d.tgt_inflection.map({"NNS": "Noun", "VBZ":
"Verb"}) + " [" + d.tgt_allomorph.map(C.ALLOMORPH_IPA) + "]")
s4 = C.rank_summary(grid, ["model", "src", "tgt"])
C.save_table(s4, "table3_fig4_allomorphs")
labels = ["Noun [s]", "Noun [z]", "Noun [ɪz]", "Verb [s]", "Verb [z]", "Verb [ɪz]"]
fig, axs = plt.subplots(1, 2, figsize=(14, 5.5))
norm = LogNorm(vmin=0.05, vmax=100)
for ax, model in zip(axs, C.MODELS):
    s = s4[s4.model == model].pivot(index="src", columns="tgt",
values="mean_rank").reindex(index=labels, columns=labels)
    paper = C.PAPER_FIG4[model].reindex(index=labels, columns=labels)
    annot = s.map(C.fmt_rank) + "\n(" + paper.map(lambda v: f"{v:g}" if
np.isfinite(v) else "–") + ")"
    im = C.heatmap(ax, s, annot, norm, title=model, fontsize=8, ylabel="Source" if
model == C.MODELS[0] else "")
    ax.tick_params(axis="x", rotation=45)
fig.colorbar(im, ax=axs, label="mean rank (log)", shrink=0.8)
fig.suptitle("Figure 3. Paper Fig. 4: transfer across allomorphs — ours (paper; – =
not in the paper)", x=0.45, y=1.0)
C.save_fig(fig, "fig03_fig4_allomorphs")
plt.show()
```

Figure 3. Paper Fig. 4: transfer across allomorphs — ours (paper; – = not in the paper)

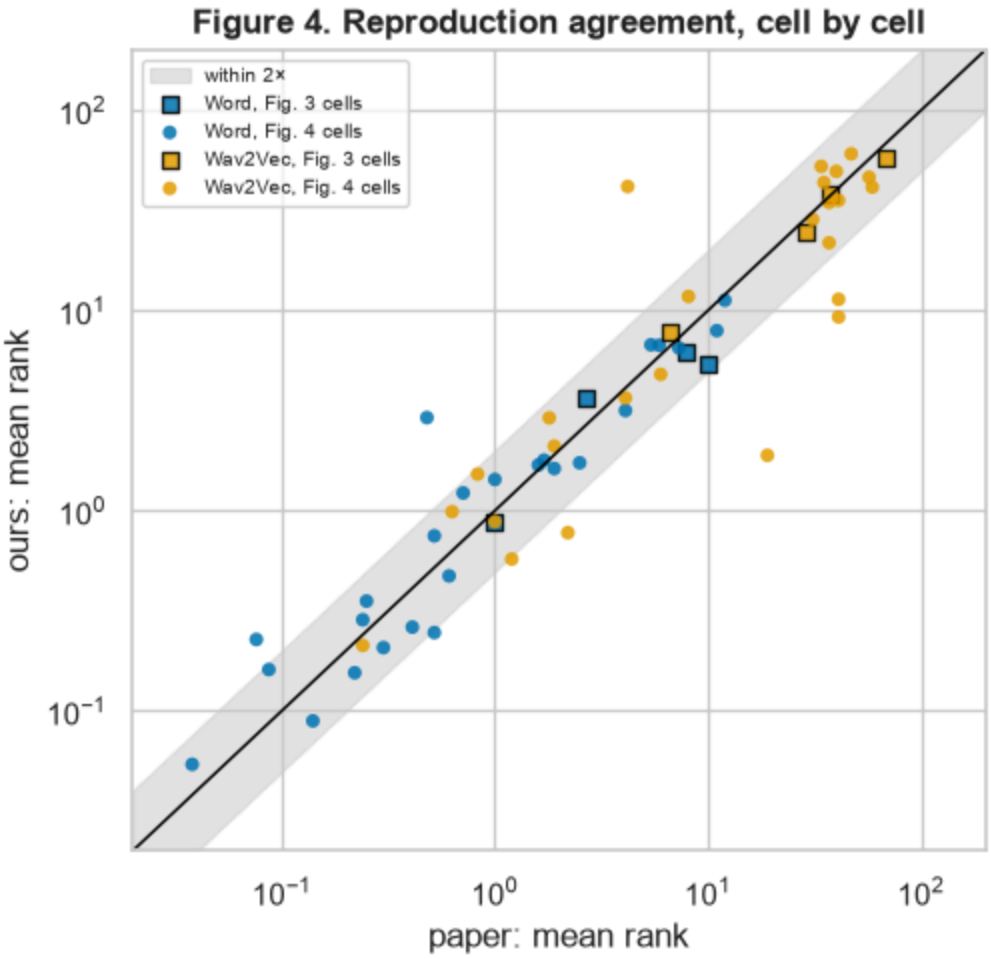


In [7]:

```
pts = []
for _, r in s3.iterrows():
    pts.append({"model": r.model, "figure": "Fig. 3", "cell": f"{r.src_inflection}
→{r.tgt_inflection}", "paper": r.paper_mean_rank, "ours": r.mean_rank})
for _, r in s4.iterrows():
    if r.src in C.PAPER_FIG4[r.model].index and r.tgt in
C.PAPER_FIG4[r.model].columns:
        pts.append({"model": r.model, "figure": "Fig. 4", "cell": f"{r.src}
→{r.tgt}", "paper": C.PAPER_FIG4[r.model].loc[r.src, r.tgt], "ours": r.mean_rank})
pts = pd.DataFrame(pts)
pts["ratio"] = pts.ours / pts.paper
C.save_table(pts, "table4_reproduction_agreement")

fig, ax = plt.subplots(figsize=(5.5, 5.2))
lim = (0.02, 200)
xs = np.geomspace(*lim, 50)
ax.fill_between(xs, xs / 2, xs * 2, color="#cccccc", alpha=0.5, label="within 2×")
ax.plot(xs, xs, color="black", lw=1)
for model in C.MODELS:
    for figname, marker in [("Fig. 3", "s"), ("Fig. 4", "o")]:
        d = pts[(pts.model == model) & (pts.figure == figname)]
        ax.scatter(d.paper, d.ours, marker=marker, s=40 if marker == "s" else 25,
color=C.MODEL_COLORS[model],
                   edgecolor="black" if marker == "s" else "none", label=f"{model},
{figname} cells", alpha=0.85)
ax.set(xscale="log", yscale="log", xlim=lim, ylim=lim, xlabel="paper: mean rank",
ylabel="ours: mean rank")
within = pts.assign(w=lambda d: d.ratio.between(0.5,
2)).groupby("model").w.agg(["sum", "size"])
ax.set_title("Figure 4. Reproduction agreement, cell by cell")
ax.legend(fontsize=7, loc="upper left")
C.save_fig(fig, "fig04_reproduction_agreement")
plt.show()
display(Markdown("Cells within 2× of the paper: " + "; ".join(f"**{m}**
{r['sum']}/{r['size']}" for m, r in within.iterrows())
```

```
                + f". Median ratio ours/paper: " + "; ".join(f"{m} {v:.2f}" for m,
v in pts.groupby('model').ratio.median().items()) + "."))
```



Cells within 2× of the paper: **Wav2Vec** 23/29; **Word** 26/29. Median ratio ours/paper: Wav2Vec 0.89; Word 0.94.

In [8]:
```
# medians next to means: the paper's median-rank figure (its Fig. 13) is all 0-3
med = s4.pivot_table(index="src", columns=["model", "tgt"],
values="median_rank").reindex(labels)
C.save_table(med.reset_index(), "table5_fig4_medians")
med.loc[:, C.MODELS].round(1)
```

Out[8]:

model							Word					Wav2Vec
tgt	Noun [s]	Noun [z]	Noun [ɪz]	Verb [s]	Verb [z]	Verb [ɪz]	Noun [s]	Noun [z]	Noun [ɪz]	Verb [s]	Verb [z]	Verb [ɪz]
src												
Noun [s]	0.0	0.0	0.0	0.0	0.0	2.0	0.0	0.0	0.0	0.0	2.0	9.0
Noun [z]	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	2.0	8.0
Noun [ɪz]	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	2.0	4.0	3.0
Verb [s]	0.0	0.0	0.0	0.0	0.0	2.0	0.0	0.0	0.0	0.0	0.0	1.0
Verb [z]	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
Verb [ɪz]	0.0	0.0	0.0	1.0	1.0	1.5	0.0	0.0	0.0	0.0	0.0	4.0

What this shows.

- **The reproduction holds.** All 8 Fig. 3 cells and 41 of 50 Fig. 4 cells are within 2× of the paper (probe 22/25, wav2vec 19/25). The median ratio of ours to the paper's is 0.94 for the probe and 0.89 for wav2vec.
- **The qualitative pattern matches.** The word probe transfers almost as well across noun ↔ verb, and across [s]/[z]/[ɪz], as within. It shows the paper's same weak spots: the *Verb [z]* target column and the *Noun [ɪz]* source row. Raw wav2vec is much more sensitive to both, e.g. NNS → VBZ is 58 vs. 5.4 for the probe.
- **Wav2vec's cells disagree with the paper more** (e.g. Noun [s] → Noun [s]: 9.3 vs. 41), because its means are driven by a few failed trials. Medians are 0 in most cells for both models, and top-1 accuracy is 0.77–0.87 in every Fig. 3 cell except wav2vec NNS → VBZ (0.34).

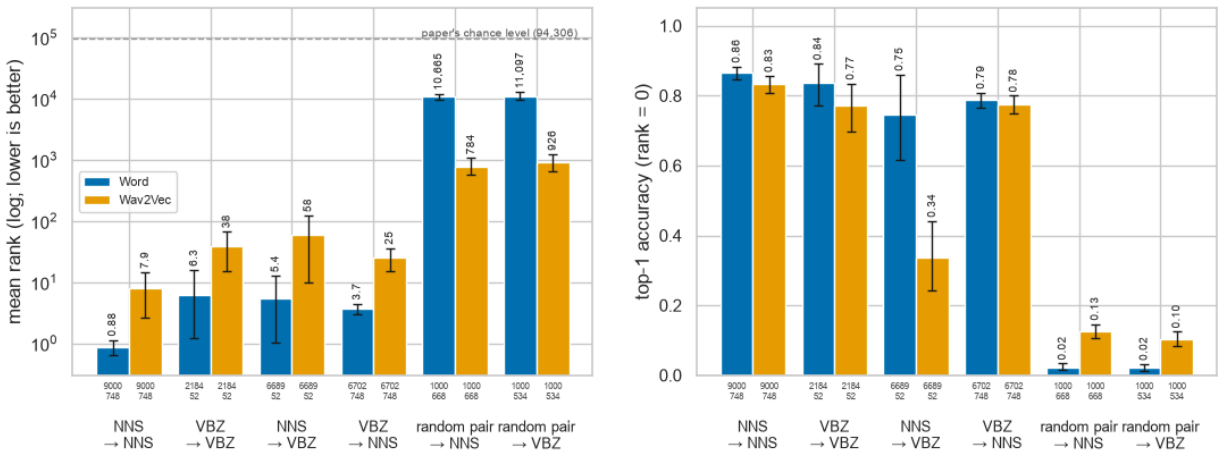
3. Baselines

How much better than "anything" are these analogies? The random-pair control takes the difference vector from a random pair of words (instead of a noun or verb inflection) and applies it to the same target bases.

- Left panel: mean rank (log scale, 95% CI).
- Right panel: top-1 accuracy.
- The small numbers under each bar are *n trials / n target words*.

In [9]:
```
conds = {
    "NNS\n→ NNS": u_excl[(u_excl.src_inflection == "NNS") & (u_excl.tgt_inflection
== "NNS")],
    "VBZ\n→ VBZ": u_excl[(u_excl.src_inflection == "VBZ") & (u_excl.tgt_inflection
== "VBZ")],
    "NNS\n→ VBZ": u_excl[(u_excl.src_inflection == "NNS") & (u_excl.tgt_inflection
== "VBZ")],
    "VBZ\n→ NNS": u_excl[(u_excl.src_inflection == "VBZ") & (u_excl.tgt_inflection
== "NNS")],
    "random pair\n→ NNS": results[results.experiment == "random_to_NNS"],
    "random pair\n→ VBZ": results[results.experiment == "random_to_VBZ"],
}
base = pd.concat([d.assign(condition=k) for k, d in conds.items()],
ignore_index=True)
s5 = C.rank_summary(base, ["model", "condition"])
C.save_table(s5.assign(condition=s5.condition.str.replace("\n", " ")),
"table6_baselines")
order = list(conds)
fig, axs = plt.subplots(1, 2, figsize=(13, 4.2))
C.ci_bars(axs[0], s5, "condition", order, log=True, ylabel="mean rank (log; lower
is better)")
axs[0].axhline(C.PAPER_CHANCE_RANK, color=C.CONTROL_COLOR, ls="--", lw=1)
axs[0].text(len(order) - 0.5, C.PAPER_CHANCE_RANK, "paper's chance level (94,306)",
ha="right", va="bottom", fontsize=7, color="#555555")
axs[0].set_ylim(0.3, 3e5)
C.ci_bars(axs[1], s5, "condition", order, est="top1", lo="top1_lo", hi="top1_hi",
ylabel="top-1 accuracy (rank = 0)", fmt="{:.2f}")
axs[1].set_ylim(0, 1.05)
for ax in axs:
    ax.tick_params(axis="x", labelsize=9)
axs[0].legend(fontsize=8)
fig.suptitle("Figure 5. Real analogies vs. the random-pair control")
C.save_fig(fig, "fig05_baselines")
plt.show()
s5.assign(condition=s5.condition.str.replace("\n", " "))[["model", "condition",
"mean_rank", "mean_lo", "mean_hi", "median_rank", "top1", "n_trials",
"n_items"]].style.hide(axis="index").format(precision=2)
```

Figure 5. Real analogies vs. the random-pair control



Out[9]:

model	condition	mean_rank	mean_lo	mean_hi	median_rank	top1	n_trials	n_items
Wav2Vec	NNS → NNS	7.87	2.61	14.57	0.00	0.83	9000	748
Wav2Vec	NNS → VBZ	58.07	9.84	122.20	2.00	0.34	6689	52
Wav2Vec	VBZ → NNS	24.94	15.10	35.87	0.00	0.78	6702	748
Wav2Vec	VBZ → VBZ	38.09	15.41	66.70	0.00	0.77	2184	52
Wav2Vec	random pair → NNS	783.96	572.27	1069.26	28.00	0.13	1000	668
Wav2Vec	random pair → VBZ	926.00	653.63	1237.94	35.00	0.10	1000	534
Word	NNS → NNS	0.88	0.65	1.14	0.00	0.86	9000	748
Word	NNS → VBZ	5.44	1.01	12.66	0.00	0.75	6689	52
Word	VBZ → NNS	3.69	2.95	4.45	0.00	0.79	6702	748
Word	VBZ → VBZ	6.26	1.23	15.88	0.00	0.84	2184	52
Word	random pair → NNS	10664.51	9397.43	11989.14	2096.00	0.02	1000	668
Word	random pair → VBZ	11096.58	9488.27	12783.16	1956.50	0.02	1000	534

- **Real analogies are far better than the control.** Every real analogy is orders of magnitude better than the random-pair control: about 10,700–11,100 for the probe and about 780–930 for wav2vec. Top-1 accuracy drops from 0.75–0.86 to 0.02 (probe) and 0.10–0.13 (wav2vec).
- **This control is not the paper's chance level of 94,306.** The paper defines its random baseline the same way, by sampling a random counterfactual pair, so how it got 94,306 is an open question. It does not match the papers tests.
- **Why the control is so much lower for wav2vec.** In raw wav2vec, $b - a + c$ stays close to c, and an inflected form sounds like its base. Even a random difference vector therefore lands near the target.

4. Past-tense pilot: VBD analogies

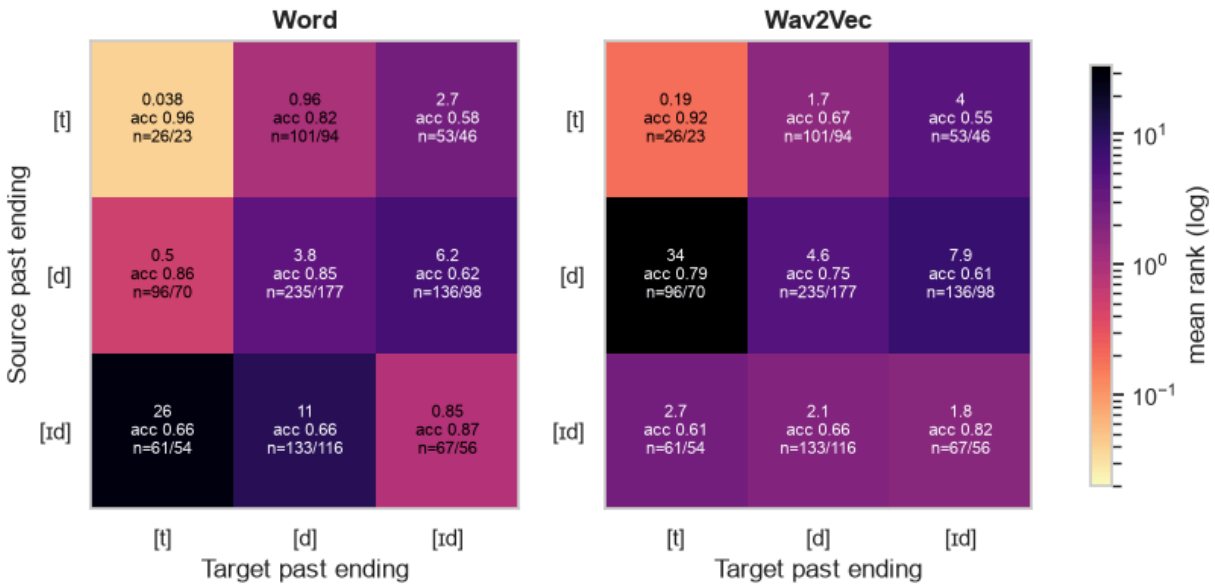
The analogy runs already contain past-tense trials (*walk : walked :: play : ?*), which the paper never summarized. We port the authors' own unpublished VBD analysis (their summary notebook, cells 69–84), without the word-frequency filter.

- **Regular verbs.** The past form is the base plus *-ed/-d/-t* (837 pairs). Source and target are both regular.
- **Pronunciation.** The aligner writes the *-ed* syllable as either `IH D` or `AH D`, in a 65 : 35 ratio (`checks/id_label_check.md`). We count both as **[ɪd]**; the authors' code silently dropped the `AH D` verbs.
- **Authors' exclusions.** We also apply the authors' exclusions: stress-shift verbs (*record, permit, …*), *do, hide, make, dream, leap*.

In [10]:
```
reg = vbd[(vbd.experiment == "regular") & (vbd.class_from == "regular") &
~vbd.authors_excluded].copy()
top3 = reg.allomorph_from_canon.value_counts().head(3).index.tolist()
assert set(top3) == {"T", "D", "IH D"}, top3
reg3 = reg[reg.allomorph_from_canon.isin(top3) &
reg.allomorph_to_canon.isin(top3)].copy()
lab = {"T": "[t]", "D": "[d]", "IH D": "[ɪd]"}
reg3["src"], reg3["tgt"] = reg3.allomorph_from_canon.map(lab),
reg3.allomorph_to_canon.map(lab)
s6 = C.rank_summary(reg3, ["model", "src", "tgt"])
C.save_table(s6, "table7_vbd_allomorph_grid")

order6 = ["[t]", "[d]", "[ɪd]"]
fig, axs = plt.subplots(1, 2, figsize=(10, 4))
norm = LogNorm(vmin=0.02, vmax=max(30, s6.mean_rank.max()))
for ax, model in zip(axs, C.MODELS):
    s = s6[s6.model == model].set_index(["src", "tgt"])
    v = s.mean_rank.unstack().reindex(index=order6, columns=order6)
    annot = pd.DataFrame([[f"{C.fmt_rank(s.loc[(a, b), 'mean_rank'])}\nacc
{s.loc[(a, b), 'top1']:.2f}\nn={s.loc[(a, b), 'n_trials']}/{s.loc[(a, b),
'n_items']}"
                          for b in order6] for a in order6], index=order6,
columns=order6)
    im = C.heatmap(ax, v, annot, norm, title=model, fontsize=8, xlabel="Target past
ending",
                   ylabel="Source past ending" if model == C.MODELS[0] else "")
fig.colorbar(im, ax=axs, label="mean rank (log)", shrink=0.9)
fig.suptitle("Figure 6. Past tense: transfer across [t] / [d] / [ɪd] (regular
verbs) — mean rank, top-1 accuracy, n trials/target verbs",
             x=0.45, y=1.04, fontsize=10)
C.save_fig(fig, "fig06_vbd_allomorph_grid")
plt.show()
```

Figure 6. Past tense: transfer across [t] / [d] / [ɪd] (regular verbs) — mean rank, top-1 accuracy, n trials/target verbs



What the pilot shows:

- **Regular past-tense transfer across [t] / [d] / [ɪd] works** (Figure 6).
  - In the probe, every cell has top-1 accuracy ≥ 0.58 (26–235 trials per cell). Within-ending cells are best (0.85–0.96), and transfer *into* [ɪd] is weakest (0.58–0.62).
  - That is the same kind of mild allomorph sensitivity the paper reports for -s.
  - The probe is at or above raw wav2vec in top-1 accuracy in all 9 cells, but the confidence intervals overlap in most of them. In this pilot the probe's advantage is smaller than for -s.